

CODE-PRIORITY

```
#include<stdio.h>
#include<stdlib.h>
typedef struct
{
    int pno;
    int pri;
    int btime;
    int wtime;
} sp;
int main()
{
    int i, j, n;
    int tbm = 0, totwtime = 0, totttime = 0;
    sp *p, t;
    printf("\nPRIORITY SCHEDULING.\n");
    printf("\nEnter the no of process....\n");
    scanf("%d", &n);
    p = (sp*)malloc(sizeof(sp) * n);
    printf("Enter the burst time and priority:\n");
    for(i = 0; i < n; i++)
    {
        printf("Process %d:", i + 1);
        scanf("%d%d", &p[i].btime, &p[i].pri);
        p[i].pno = i + 1;
        p[i].wtime = 0;
    }
    for(i = 0; i < n - 1; i++)
        for(j = i + 1; j < n; j++)
        {
            if(p[i].pri > p[j].pri)
            {
                t = p[i];
                p[i] = p[j];
                p[j] = t;
            }
        }
    printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");
```

```

for(i = 0; i < n; i++)
{
    totwtime += p[i].wtime = tbm;
    tbm += p[i].btime;
    printf("\n%d\t\t%d\t\t%d\t\t%d",
        p[i].pno,
        p[i].btime,
        p[i].wtime,
        p[i].wtime + p[i].btime);
}
totttime = tbm;
printf("\n\nTotal Waiting Time: %d", totwtime);
printf("\nAverage Waiting Time: %f", (float)totwtime / n);
printf("\nTotal Turnaround Time: %d", totttime);
printf("\nAverage Turnaround Time: %f", (float)totttime / n);
return 0;
}

```

OUTPUT:

Enter the no of process....

3

Enter the burst time and priority:

Process 1: 10 3

Process 2: 5 1

Process 3: 8 2

PRIORITY SCHEDULING.

Process	Burst Time	Waiting Time	Turnaround Time
---------	------------	--------------	-----------------

2	5	0	5
3	8	5	13
1	10	13	23

Total Waiting Time: 18

Average Waiting Time: 6.000000

Total Turnaround Time: 23

Average Turnaround Time: 7.666667

CODE-FCFS

```
#include <stdio.h>
#include <stdlib.h>
struct fcfs
{
    int pid;
    int btime;
    int wtime;
    int ttime;
} p[10];
int main()
{
    int i, n;
    int totwtime = 0, totttime = 0;
    printf("\nFCFS Scheduling...\n");
    printf("Enter the number of processes:
");
    scanf("%d", &n);
    for(i = 0; i < n; i++)
    {
        p[i].pid = i + 1;
        printf("Burst time of process %d: ", i +
1);
        scanf("%d", &p[i].btime);
    }
    p[0].wtime = 0;
    p[0].ttime = p[0].btime;
    for(i = 1; i < n; i++)
    {
        p[i].wtime = p[i-1].wtime + p[i-
1].btime;
        p[i].ttime = p[i].wtime + p[i].btime;
    }
    for(i = 0; i < n; i++)
    {
        totwtime += p[i].wtime;
        totttime += p[i].ttime;
    }
    printf("\nPROCESS\tBURST
TIME\tWAITING TIME\tTURNAROUND
TIME\n");
```

```
for(i = 0; i < n; i++)
{
    printf("%d\t\t%d\t\t%d\t\t%d\n",
        p[i].pid,
        p[i].btime,
        p[i].wtime,
        p[i].ttime);
}
printf("\nTotal waiting time: %d",
totwtime);
printf("\nAverage waiting time: %f",
(float)totwtime / n);
printf("\nTotal turnaround time: %d",
totttime);
printf("\nAverage turnaround time:
%f\n", (float)totttime / n);

return 0;
}
```

OUTPUT:

```
Enter the number of processes: 3
Burst time of process 1: 5
Burst time of process 2: 3
Burst time of process 3: 8
```

FCFS Scheduling...

PROCESS	BURST TIME	WAITING TIME	TURNAROUND TIME
1	5	0	5
2	3	5	8
3	8	8	16

```
Total waiting time: 13
Average waiting time: 4.333333
Total turnaround time: 29
Average turnaround time: 9.666667
```

CODE-ROUND ROBIN

```
#include<stdio.h>
int TRUE = 0;
int FALSE = -1;
int tbt[30], bt[30], tat[30], n = 0, wt[30], qt = 0, tqt = 0;
int time = 0, lmore, t_tat = 0, t_wt = 0;
void main()
{
    int i;
    printf("\nEnter no. of processors:");
    scanf("%d", &n);
    printf("\nEnter Quantum Time:");
    scanf("%d", &qt);
    for(i = 0; i < n; i++)
    {
        printf("\nEnter Burst Time of Processor[%d]:", i + 1);
        scanf("%d", &bt[i]);
        tbt[i] = bt[i];
        wt[i] = tat[i] = 0;
    }
    lmore = TRUE;
    while(lmore == TRUE)
    {
        lmore = FALSE;
        for(i = 0; i < n; i++)
        {
            if(bt[i] != 0)
            {
                wt[i] = wt[i] + (time - tat[i]);
                tqt = 1;
                while(tqt <= qt && bt[i] != 0)
                {
                    lmore = TRUE;
                    bt[i] = bt[i] - 1;
                    tqt++;
                    time++;
                    tat[i] = time;
                }
            }
        }
    }
}
```

```

        }
    }
}
printf("\nProcessor ID\tBurstTime\tTurnAroundTime\tWaitingTime\n");
for(i = 0; i < n; i++)
{
    printf("Processor%d\t\t%d\t\t%d\t\t%d\n",
        i + 1, tbt[i], tat[i], wt[i]);
    t_tat = t_tat + tat[i];
    t_wt = t_wt + wt[i];
}
printf("\nTotal Turn Around Time:%d", t_tat);
printf("\nAverage Turn Around Time:%d", t_tat / n);
printf("\nTotal Waiting Time:%d", t_wt);
printf("\nAverage Waiting Time:%d", t_wt / n);
}

```

OUTPUT:

Enter no. of processors: 3

Enter Quantum Time: 2

Enter Burst Time of Processor[1]: 5

Enter Burst Time of Processor[2]: 4

Enter Burst Time of Processor[3]: 2

Processor ID	BurstTime	TurnAroundTime	WaitingTime
Processor1	5	11	6
Processor2	4	10	6
Processor3	2	6	4

Total Turn Around Time:27

Average Turn Around Time:9

Total Waiting Time:16

Average Waiting Time:5

CODE-SENDER

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/shm.h>
#include<string.h>
int main()
{
    int i;
    void *shared_memory;
    char buff[100];
    int shmid;
    shmid = shmget((key_t)2345, 1024, 0666|IPC_CREAT);
    printf("Key of shared memory is %d\n", shmid);
    shared_memory = shmat(shmid, NULL, 0);
    printf("Process attached at %p\n", shared_memory);
    printf("Enter some data to write to shared memory\n");
    read(0, buff, 100);
    strcpy(shared_memory, buff); // data written to shared memory
    printf("You wrote : %s\n", (char *)shared_memory);
}
```

CODE-RECEIVER

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/shm.h>
#include<string.h>
int main()
{
    int i;
    void *shared_memory;
    char buff[100];
    int shmid;
    shmid = shmget((key_t)2345, 1024, 0666);
    printf("Key of shared memory is %d\n", shmid);
    shared_memory = shmat(shmid, NULL, 0);
    printf("Process attached at %p\n", shared_memory);
    printf("Data read from shared memory is : %s\n", (char*)shared_memory);
}
```

OUTPUT:

```
student@pec-H81:~$ gcc sender.c
student@pec-H81:~$ ./a.out
Key of shared memory is 2
Process attached at 0x71a89069a000
Enter some data to write to shared memory
hi
You wrote : hi
```

```
student@pec-H81:~$ gcc reciever.c
student@pec-H81:~$ ./a.out
Key of shared memory is 2
Process attached at 0x7a49f3a6e000
Data read from shared memory is : hi
```

CODE:

```
#include<stdio.h>
int mutex=1,full=0,empty=5,x=0;
main()
{
int n;
void producer();
void consumer();
int wait(int);
int signal(int);
printf("\n PRODUCER & CONSUMER EX1\n");
while(1)
{
printf("\n ENTER YOUR CHOICE :");
scanf("%d",&n);
switch(n)
{
case 1:
if((mutex==1)&&(empty!=0))
{
producer();
mutex=wait(mutex);
full=signal(full);
empty=wait(empty);
mutex=signal(mutex);
}
else
printf("BUFFER IS FULL");
break;
case 2:
if((mutex==1)&&(full!=0))
{
consumer();
mutex=wait(mutex);
full=wait(full);
empty=signal(empty);
mutex=signal(mutex);
}
else
printf("BUFFER IS EMPTY");
break;
case 3:
exit(0);
break;
}
}
```

```

}
int wait(int s)
{
return(--s);
}
int signal(int s)
{
return(++s);
}
void producer()
{
x++;
printf("\n producer produces the
item%d",x);
}
void consumer()
{
printf("\n consumer consumes item%d",x);
x--;
}
```

OUTPUT

```
1. PRODUCER
2. CONSUMER
3. EXIT
ENTER YOUR CHOICE
1
producer produces the item1
ENTER YOUR CHOICE
1
producer produces the item2
ENTER YOUR CHOICE
1
producer produces the item3
ENTER YOUR CHOICE
1
BUFFER IS FULL
ENTER YOUR CHOICE
2
consumer consumes item3
ENTER YOUR CHOICE
2
consumer consumes item2
ENTER YOUR CHOICE
```

CODE:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define N 5
sem_t room;
sem_t chopstick[N];
sem_t mutex;
void* philosopher(void* num)
{
    int id = *(int*)num;
    while (1)
    {
        // Thinking
        printf("Philosopher %d is thinking\n", id);
        sleep(1);
        sem_wait(&room);
        sem_wait(&chopstick[id]);
        sem_wait(&chopstick[(id + 1) % N]);
        sem_wait(&mutex);
        printf("Philosopher %d is eating\n", id);
        sem_post(&mutex);
        sleep(2);
        sem_post(&chopstick[id]);
        sem_post(&chopstick[(id + 1) % N]);
        sem_post(&room);
    }
}
int main()
{
    pthread_t tid[N];
    int phil[N];
    sem_init(&room, 0, N - 1);
    sem_init(&mutex, 0, 1);
    for (int i = 0; i < N; i++)
        sem_init(&chopstick[i], 0, 1);
```

```
    for (int i = 0; i < N; i++)
    {
        phil[i] = i;
        pthread_create(&tid[i], NULL,
            philosopher, &phil[i]);
    }
    for (int i = 0; i < N; i++)
        pthread_join(tid[i], NULL);
    return 0;
}
```

OUTPUT

Philosopher 0 is thinking
Philosopher 1 is thinking
Philosopher 2 is thinking
Philosopher 3 is thinking
Philosopher 4 is thinking

Philosopher 0 is eating
Philosopher 2 is eating

Philosopher 0 is thinking
Philosopher 1 is eating

Philosopher 2 is thinking
Philosopher 3 is eating

Philosopher 1 is thinking
Philosopher 4 is eating

Philosopher 3 is thinking
Philosopher 0 is eating

Philosopher 4 is thinking
Philosopher 2 is eating